

Day 9

SPI Master:

Verilog Code:

```
module spi_master #(
    parameter CLK_DIV = 4, // SCLK = clk / (2 * CLK_DIV)
    parameter WIDTH = 8
)(
    input clk, rst,
    input start,
    input [WIDTH-1:0] mosi_data,
    output reg [WIDTH-1:0] miso_data,
    output reg mosi,
    input miso,
    output reg sclk,
    output reg cs_n,
    output reg done
);

    localparam IDLE    = 2'b00;
    localparam TRANSFER = 2'b01;
    localparam FINISH  = 2'b10;

    reg [1:0] state;
    reg [$clog2(CLK_DIV):0] clk_cnt;
    reg [3:0] bit_cnt; // Supports up to 16-bit
    reg [WIDTH-1:0] shift_tx;
    reg [WIDTH-1:0] shift_rx;

    always @(posedge clk) begin
        if (rst) begin
            state <= IDLE;
            sclk <= 0;
            cs_n <= 1;
            mosi <= 0;
            done <= 0;
            clk_cnt <= 0;
            bit_cnt <= 0;
            shift_tx <= 0;
            shift_rx <= 0;
            miso_data <= 0;
        end else begin
            done <= 0;
            case (state)
                IDLE: begin
                    sclk <= 0;
                    cs_n <= 1;
                    clk_cnt <= 0;
                    bit_cnt <= 0;
                    if (start) begin
                        cs_n <= 0;
                        // Pre-load MOSI with MSB and buffer the rest
                        mosi <= mosi_data[WIDTH-1];
                        shift_tx <= {mosi_data[WIDTH-2:0], 1'b0};
                        state <= TRANSFER;
                    end
                end
            endcase
        end
    end
```

```

end

TRANSFER: begin
  if (clk_cnt == CLK_DIV - 1) begin
    clk_cnt <= 0;
    sclk <= ~sclk; // Toggle SCLK

    if (!sclk) begin
      // RISING EDGE: Sample MISO
      shift_rx <= {shift_rx[WIDTH-2:0], miso};
    end else begin
      // FALLING EDGE: Shift out next MOSI bit
      if (bit_cnt == WIDTH - 1) begin
        state <= FINISH;
      end else begin
        bit_cnt <= bit_cnt + 1;
        mosi <= shift_tx[WIDTH-1];
        shift_tx <= {shift_tx[WIDTH-2:0], 1'b0};
      end
    end
  end else begin
    clk_cnt <= clk_cnt + 1;
  end
end

FINISH: begin
  // Finalize the last clock half-period
  if (clk_cnt == CLK_DIV - 1) begin
    sclk <= 0;
    cs_n <= 1;
    miso_data <= shift_rx;
    done <= 1;
    state <= IDLE;
  end else begin
    clk_cnt <= clk_cnt + 1;
  end
end

default: state <= IDLE;
endcase
end
end
endmodule

```

Testbench:

```

`timescale 1ns/1ps
module tb;
  reg    clk, rst, start;
  reg [7:0] mosi_data;
  wire [7:0] miso_data;
  wire    mosi, sclk, cs_n, done;
  wire    miso;

  // Loopback — connect MOSI back to MISO
  assign miso = mosi;

  spi_master #(.CLK_DIV(4), .WIDTH(8)) uut(

```

```

.clk(clk), .rst(rst),
.start(start),
.mosi_data(mosi_data),
.miso_data(miso_data),
.mosi(mosi),
.miso(miso),
.sclk(sclk),
.cs_n(cs_n),
.done(done)
);

initial begin clk = 0; forever #5 clk = ~clk; end

initial begin
    $dumpfile("spi.vcd");
    $dumpvars(0, tb);
end

task spi_transfer(input [7:0] data);
begin
    wait(!cs_n == 0); // wait for idle
    @(negedge clk);
    start = 1;
    mosi_data = data;
    @(negedge clk);
    start = 0;

    wait(done);
    @(posedge clk);

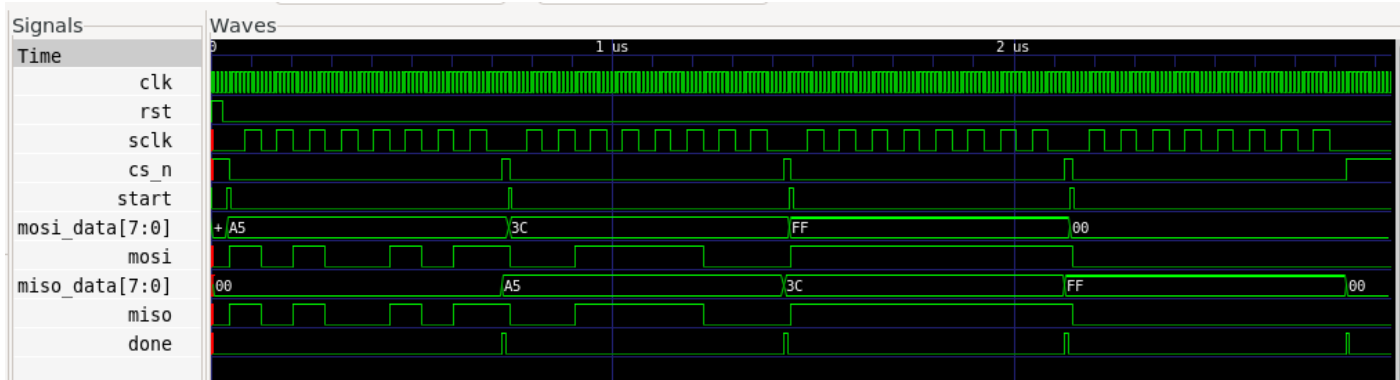
    if(miso_data == data)
        $display("PASS: sent %h received %h", data, miso_data);
    else
        $display("FAIL: sent %h received %h", data, miso_data);
end
endtask

initial begin
    rst = 1; start = 0; miso_data = 0;
    #30; rst = 0; #10;

    spi_transfer(8'hA5);
    spi_transfer(8'h3C);
    spi_transfer(8'hFF);
    spi_transfer(8'h00);

    #100; $finish;
end
endmodule

```



Simulation and Synthesis output:

VCD info: dumpfile spi.vcd opened for output.

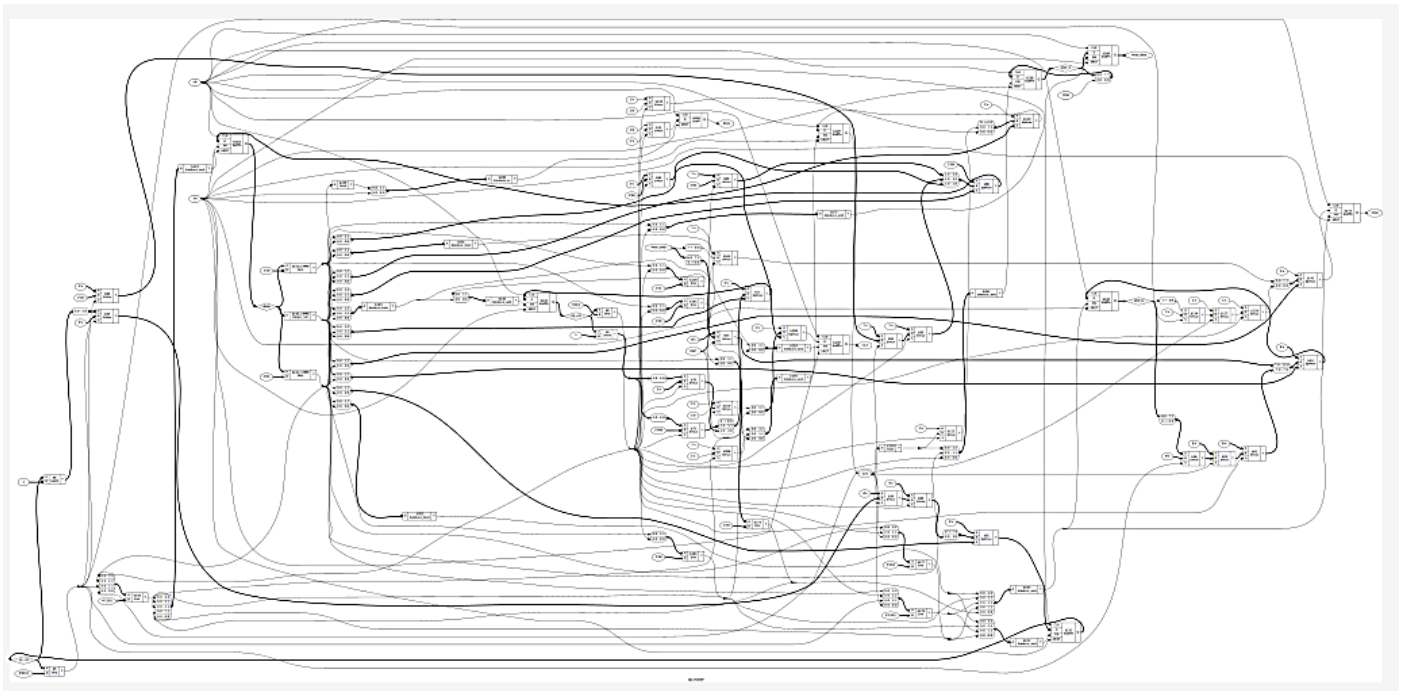
PASS: sent a5 received a5

PASS: sent 3c received 3c

PASS: sent ff received ff

PASS: sent 00 received 00

spi_tb.v:59: \$finish called at 2935000 (1ps)



2.26. Printing statistics.

=== spi_master ===

Number of wires:	107
Number of wire bits:	144
Number of public wires:	15
Number of public wire bits:	50
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	131
\$_ANDNOT_	35
\$_DFF_P_	3
\$_MUX_	10
\$_NAND_	5
\$_NOR_	2
\$_NOT_	4

\$_ORNOT_	8
\$_OR_	25
\$_SDFFE_PP0P_	32
\$_SDFFE_PP1P_	1
\$_SDFF_PPO_	1
\$_XNOR_	2
\$_XOR_	3